

Лабораторна робота №7

Звіт

З дисципліни “Сучасні інтернет технології”
на тему: “Пагінація у застосунку ASP.NET Core”.

Студента 4 курсу: Групи МІТ-41
Демиденко Андрій

Київ - 2025р.

Хід виконання роботи

Тема: Реалізація пагінації даних та наповнення бази даних у веб-застосунку ASP.NET Core

У ході виконання лабораторної роботи було поставлено завдання розширити функціонал існуючого веб-застосунку, додавши нову предметну область «Автосалон» та реалізувавши механізм посторінкового виведення даних (пагінацію). Це необхідно для оптимізації роботи з великими масивами даних, щоб уникнути завантаження всіх записів з бази даних одночасно, що могло б негативно вплинути на продуктивність сервера та користувацький досвід. Робота розпочалася зі створення нової сутності Car у проєкті WebApplication.Data. Клас містить властивості, що описують автомобіль: марка, модель, рік випуску та ціна. Для поля ціни було явно вказано тип даних SQL decimal(18,2) за допомогою атрибутів даних, щоб уникнути втрати точності при збереженні фінансових значень. Після створення моделі та додавання відповідного DbSet у контекст бази даних, було створено та застосовано нову міграцію, що оновила схему бази даних SQL Server.

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WebApplication.Data.Models
{
    15 references
    public class Car
    {
        [Key]
        0 references
        public int Id { get; set; }

        [Required]
        [Display(Name = "Марка")]
        10 references
        public string Brand { get; set; } = string.Empty;

        [Required]
        [Display(Name = "Модель")]
        10 references
        public string Model { get; set; } = string.Empty;

        [Display(Name = "Рік випуску")]
        10 references
        public int Year { get; set; }

        [Display(Name = "Ціна")]
        [Column(TypeName = "decimal(18,2)")]
        10 references
        public decimal Price { get; set; }
    }
}
```

Наступним кроком стала реалізація логіки пагінації. Для цього в шарі даних було створено допоміжний узагальнений клас `PaginatedList<T>`, який наслідує стандартний `List<T>`. Цей клас містить логіку розрахунку загальної кількості сторінок (`TotalPages`) та властивості для визначення наявності попередньої або наступної сторінки. Ключовим елементом реалізації є метод `CreateAsync`, який приймає запит `IQueryable`, номер сторінки та розмір сторінки. Використовуючи методи Entity Framework `Skip` (пропустити певну кількість записів) та `Take` (взяти фіксовану кількість), цей клас забезпечує завантаження з бази даних лише тих записів, які необхідні для відображення поточної сторінки. Це значно зменшує навантаження на пам'ять та мережу.

```
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WebApplication.Data.Models
{
    4 references
    public class PaginatedList<T> : List<T>
    {
        3 references
        public int PageIndex { get; private set; }
        2 references
        public int TotalPages { get; private set; }

        1 reference
        public PaginatedList(List<T> items, int count, int pageIndex, int pageSize)
        {
            PageIndex = pageIndex;
            TotalPages = (int)Math.Ceiling(count / (double)pageSize);

            this.AddRange(items);
        }

        0 references
        public bool HasPreviousPage => PageIndex > 1;
        0 references
        public bool HasNextPage => PageIndex < TotalPages;

        1 reference
        public static async Task<PaginatedList<T>> CreateAsync(IQueryable<T> source, int pageIndex, int pageSize)
        {
            var count = await source.CountAsync();
            var items = await source.Skip((pageIndex - 1) * pageSize).Take(pageSize).ToListAsync();
            return new PaginatedList<T>(items, count, pageIndex, pageSize);
        }
    }
}
```

Для демонстрації роботи пагінації необхідно було мати достатню кількість даних у системі. Щоб не вводити дані вручну щоразу при запуску, було реалізовано механізм автоматичного наповнення бази даних (Data Seeding). Було створено клас `DbInitializer`, який перевіряє, чи порожня таблиця автомобілів. Якщо записів немає, він автоматично додає масив з 10 автомобілів японського виробництва (Toyota, Honda, Nissan тощо). Виклик цього ініціалізатора було додано у файл `Program.cs` на етапі запуску застосунку, що гарантує наявність тестових даних одразу після старту сервера.

```

namespace WebApplication.Data.Data
{
    1 reference
    public static class DbInitializer
    {
        1 reference
        public static void Initialize(ApplicationDbContext context)
        {
            if (context.Cars.Any())
            {
                return;
            }

            var cars = new Car[]
            {
                new Car { Brand = "Toyota", Model = "Corolla", Year = 2022, Price = 21500 },
                new Car { Brand = "Honda", Model = "Civic", Year = 2023, Price = 24600 },
                new Car { Brand = "Nissan", Model = "GT-R", Year = 2021, Price = 113500 },
                new Car { Brand = "Mazda", Model = "MX-5 Miata", Year = 2023, Price = 28000 },
                new Car { Brand = "Subaru", Model = "Impreza", Year = 2022, Price = 19700 },
                new Car { Brand = "Mitsubishi", Model = "Outlander", Year = 2023, Price = 27500 },
                new Car { Brand = "Lexus", Model = "RX 350", Year = 2023, Price = 48500 },
                new Car { Brand = "Suzuki", Model = "Swift", Year = 2022, Price = 16500 },
                new Car { Brand = "Infiniti", Model = "Q50", Year = 2023, Price = 42600 },
                new Car { Brand = "Acura", Model = "Integra", Year = 2024, Price = 31500 }
            };

            context.Cars.AddRange(cars);

            context.SaveChanges();
        }
    }
}

```

Далі було розроблено контролер CarsController, який використовує створений раніше репозиторій IWebAppRepository. У методі Index контролер отримує запит на вибірку всіх автомобілів, але не виконує його одразу. Замість цього запит передається у PaginatedList, де відбувається фільтрація за сторінками. Розмір сторінки було встановлено на рівні 5 записів, що при наявності 10 автомобілів дозволяє продемонструвати перехід між двома сторінками. Також було створено представлення Index.cshtml, яке виводить дані у вигляді таблиці та містить кнопки навігації «Попередня» та «Наступна», активність яких залежить від поточного номера сторінки.

```

1 reference
public async Task<IActionResult> Index(int? pageNumber)
{
    var carsQuery = _repository.ReadAll<Car>();

    int pageSize = 3;

    var paginatedCars = await PaginatedList<Car>.CreateAsync(carsQuery, pageNumber ?? 1, pageSize);

    return View(paginatedCars);
}

```

На завершальному етапі було створено сторінку для додавання нових автомобілів (Create.cshtml) та налаштовано маршрутизацію. Це дозволило перевірити роботу системи в комплексі: після автоматичного наповнення бази даних користувач бачить перші 5 автомобілів, може перейти на другу сторінку, щоб побачити решту, або додати нове авто, яке, залежно від кількості записів, з'явиться на відповідній сторінці. Реалізація пагінації на рівні бази даних забезпечила масштабованість рішення для роботи з великими обсягами інформації.

[Додати авто](#)

Марка	Модель	Рік	Ціна (\$)
Toyota	Corolla	2022	21 500
Honda	Civic	2023	24 600
Nissan	GT-R	2021	113 500

[« Попередня](#) [Сторінка 1 з 4](#) [Наступна »](#)

Висновок

У ході виконання лабораторної роботи було успішно розширено функціональність веб-застосунку ASP.NET Core, побудованого за архітектурою модульного моноліту. Головним результатом стала реалізація механізму пагінації (посторінкового виведення даних), що є критично важливим для оптимізації роботи з великими масивами інформації.

Для цього було спроектовано нову сутність предметної області Car, налаштовано точність збереження фінансових даних (decimal) та оновлено структуру бази даних за допомогою міграцій Entity Framework Core. Реалізація допоміжного класу PaginatedList<T> дозволила ефективно виконувати вибірку даних безпосередньо на рівні бази даних, використовуючи методи LINQ Skip та Take над IQueryable. Такий підхід запобігає завантаженню зайвих даних у пам'ять сервера, значно підвищуючи продуктивність застосунку.

Додатково було реалізовано патерн Data Seeding (автоматичне наповнення) через клас DbInitializer, що дозволило автоматизувати процес внесення тестових даних при старті програми. Інтеграція нової логіки з наявним патерном Repository та створення відповідного інтерфейсу користувача забезпечили зручну навігацію по записах автопарку. Отримані навички дозволяють створювати масштабовані та зручні для користувача веб-інтерфейси.